PRD v0.1 — Sand Hauling Logistics Planner

Product vision

A planning tool to **visualize and manage sand-hauling fleet allocations across multiple fracs**, with an interactive **Gantt schedule** and a **daily allocation grid**, supporting rapid replans when fracs shift, pause, or extend.

The system must:

- Let you **drag/drop frac schedules** and instantly see **fleet impacts**
- Maintain **Baseline / Forecast / Actual** plans + **Sandbox scenarios**
- Track **daily truck needs** and **hauler capacity/commitments**
- **Flag conflicts** (over capacity, under supplied) and operational pain points
- Support **day-level planning** (2x12 shifts) while still allowing optional shift-level overrides later

---

1) Users & permissions

Primary user (Planner)

- Full edit rights on Forecast and Sandbox scenarios
- Can record Actuals

Viewer / Sandbox user

- Can view Baseline / Forecast / Actual
- Can create a Sandbox scenario from Forecast and make changes **without affecting the Actual Plan**
- Cannot edit Baseline directly

Since you're single-user operationally, this can be implemented as "roles" without heavy access-control overhead.

---

2) Core planning concepts and scope

Planning unit

- Primary unit: **trucks per shift** (integer)
- Default planning granularity: **per day**, assuming day & night shift are the same unless overridden
- Shifts: **2x12 hr**

Time horizon

- **Quarterly operational view** (primary)
- **Year Gantt view** (secondary)

Lanes ("fleets")

- Lanes represent "active fleets servicing pads" (labels like EVO1 don't matter and may change)
- Each frac is generally assigned to a lane and usually stays there once started
- Lanes have sequential fracs with **transition gaps** (rig-up/move/etc)

Schedule changes

You want both:

- **Chain mode:** moving a frac pushes/pulls downstream fracs in that lane, preserving transition gaps
- **Independent/overlap mode:** moving a frac does *not* move the others, allowing overlap (but should flag as a risk/conflict if needed)

---

3) Data model (the backbone)

Here's the minimal object model that supports everything you described.

A) Frac job

FracJob

- id
- padName (e.g., BIG177)
- laneId (fleet lane)
- plannedStartDateTime (or date)

- plannedEndDateTime (or computed from stages/day + total stages)
- transitionDaysAfter (or “gap to next”)
- status: Planned / Active / Paused / Complete
- customer (optional)
- basin/area (optional)
- notes, contacts

SandPlan (attached to FracJob)

- stagesPerDay (required)
- tonsPerStage (required)
- meshProducts (100 mesh, 40/70, etc) (optional; can be simple list for MVP)
- travelTimeHours (required-ish, based on your list)
- avgTonsPerLoad (optional but valuable)
- estTonsPerTruckPerShift (optional)
- onSiteStorageType: Silo / Kube (and capacity)
- prefillDateShift (optional)
- sandSources / transloads (optional multi-source support)

DemandPlan

- fracJobId
- date
- requiredTrucksPerShift (integer)
- (optional later) dayShiftRequired, nightShiftRequired

MVP approach: store demand as a daily series so it matches how you work today, and how your Excel table is built.

B) Hauler

Hauler

- id

- name (ET, HWE, Seven Point, etc.)
- splitAllowed (boolean; Revolution = true, many others false)
- homeArea / yard (optional)
- notes

HaulerCapacityCalendar

- haulerId
- date
- maxTrucksPerShift (integer) ✅ varies by day (call-offs, breakdowns)
- minCommittedTrucksPerShift (integer) ✅ contract minimums
- notes/reason (optional)

C) Allocation / assignment

To match your Excel workflow, use **range allocations + overrides**:

AllocationBlock

- scenarioId
- fracJobId
- haulerId
- startDate
- endDate
- trucksPerShift (integer)

AllocationOverride (optional for MVP, but super useful)

- scenarioId
- fracJobId
- haulerId
- date
- trucksPerShift (integer)
- reason/category (optional)

This gives you:

- Fast “assign ET 10 trucks/shift to BIG177 for Feb 22–Mar 3”
- Then tweak a single day when plans fall apart

D) Scenarios / versioning

Scenario

- id
- name: Baseline / Forecast / Actual / Sandbox: “What-if Mingo delays 3 days”
- parentScenarioId (for branching)
- locked (true for Baseline, optionally for Actual after close)
- createdAt, updatedAt

---

4) Calculations & business rules

Demand generation

You said you want trucks/shift as:

- a manual number you trust today
- plus an auto “recommendation” that you can compare against until it proves reliable

So we support both:

RequiredTrucksPerShift = Manual input (editable)
RecommendedTrucksPerShift = computed

Recommended trucks/shift (first pass)

We can start with a simple, explainable formula:

1. Compute tons/day
   tonsPerDay = stagesPerDay * tonsPerStage
2. Compute tons/shift (2 shifts/day)
   tonsPerShift = tonsPerDay / 2
3. If you have “tons/truck/shift” (or tons/truck), use it:

- If using tons/truck/shift:
  recommended = ceil(tonsPerShift / tonsPerTruckPerShift)
- If using tons/truck/day:
  recommended = ceil(tonsPerDay / tonsPerTruckPerDay)

4. If you *don't* have tons/truck, estimate from:

- avg tons/load
- cycle time approximation from travel time (plus load/unload/staging constants you can tune)

The key: the recommendation should always show its math so you can trust/adjust it.

Rounding rule

- **No partial trucks**. Always ceil() to nearest integer.

Conflict detection (real-time)

Hard stops and warnings per day:

Hard stop

- For any hauler/day:
  sumAssignedTrucksPerShift(hauler, day) > maxTrucksPerShift(hauler, day)
- For any frac/day:
  sumAssignedTrucksPerShift(frac, day) < requiredTrucksPerShift(frac, day)

Warning

- For any frac/day:
  sumAssignedTrucksPerShift(frac, day) == requiredTrucksPerShift(frac, day) (zero buffer)
- For haulers with splitAllowed = false:
  if assigned to >1 frac on same day/shift → warning (or hard stop, your call later)

Optional warning (cost/overtrucking)

- Assigned > required (you said you evaluate and reduce trucking to reduce cost)
- If storage modeling is included, warn when scheduled deliveries imply inventory overflow ("kube full / silos full" risk)

"Frac falls behind schedule" impact

To catch the pain point you described:

- Allow entering **actual stages completed per day** and/or **NPT hours**
- System computes "pace vs plan" and forecasts a new finish date
- If finish date slides right → it immediately impacts the lane schedule and fleet conflicts

This can be MVP-light: even a simple "% behind plan" and "predicted end date" is huge value.

---

5) UI/UX requirements (what you'll build)

Home screen: Gantt by lane

- Lanes = active fleets (EVO1/EVO10/etc)
- Bars = fracs (pad names)
- Bar label includes:
    - pad name
    - **required trucks/shift** (or range if varies)
- Drag/drop to change start date (and optionally duration)
- Toggle modes:
    - **Chain move ON/OFF**
    - "Allow overlap" (or allow always but flags conflicts)

Visual conflict overlays

- If a drag causes any day to go short or any hauler to exceed cap:
    - highlight impacted date range
    - show red markers on the timeline
    - quick tooltip: "Short 4 trucks/shift on Feb 26 (COP)"

Drill-down side panel (click a frac)

Tabs:

1. **Sand Info** (your slide fields)

2. **Demand curve** (daily required trucks/shift)
3. **Assignments by hauler** (who's covering it, how many)
4. Notes / contacts

View 2: Daily allocation grid (Excel replacement)

Very similar to your screenshot:

- Columns: dates
- Sections: one per frac ("BIG177 Needs", "Teamwork Needs"...)
- Rows inside: haulers
- Totals and Needs rows
- Conditional formatting:
    - shortfalls (hard stop)
    - overages (hard stop or warning)
    - zero-buffer (warning)

View 3: Hauler capacity view

For each hauler:

- daily max vs assigned
- min commitment vs assigned
- exceptions/call-offs notes

Scenarios UI

- Scenario dropdown: Baseline / Forecast / Actual / Sandbox...
- "Create sandbox from Forecast"
- "Compare sandbox vs forecast" (diff view)
- "Promote sandbox → Forecast" (apply changes)

---

6) Data input ("job builder")

Since you don't have a clean Excel import workflow yet, MVP should include a **Job Builder**:

Create Frac Job

- Pad name
- Lane
- Planned start date
- Planned end date *or* (total stages + stages/day)
- SandPlan fields (stages/day, tons/stage, travel time, storage type, etc.)
- Optional: paste-in area for "Sand Info" notes (POs, etc.)

Hauler setup

- Define hauler list and default trucks/shift caps
- Define min commitments
- Add day-specific capacity exceptions quickly (call-offs)

---

7) Reporting / exports

MVP exports (handy for morning calls):

- PDF export of:
    - Gantt view (selected date range)
    - Daily allocation grid (next 14 days)
- CSV export of allocations (per day, per frac, per hauler)

---

8) Technical approach (fits Cursor/Replit and your constraints)

You want:

- shared web app
- should keep working even if internet/external systems are down

Best-fit architecture:

- **Offline-first web app (PWA)** + local database in the browser
    - Works without internet

    - Can be hosted internally but still usable during outages
- Optional later: add sync or server persistence if you need access from multiple devices

Suggested stack

- Front-end: React + TypeScript
- Local data: IndexedDB (via a wrapper like Dexie) or SQLite-in-browser (heavier)
- UI: a Gantt/timeline component that supports drag/drop
- Export PDF: render HTML → PDF (client side or via a small server utility)

If "shared web app" means "multiple people need to access the same plan," we'd add a small backend + database. But even then, you can still do offline-first by caching locally and syncing.

---

9) MVP scope vs Phase 2/3

MVP (high value, manageable complexity)

- Job Builder (frac + sand plan fields)
- Hauler setup + daily capacities + min commitments
- Forecast scenario planning (Baseline + Forecast + Sandbox)
- Gantt view (drag/drop) with conflict flags
- Daily allocation grid (like your Excel)
- Basic export (PDF/CSV)

Phase 2

- Actuals tracking (stages completed/day)
- NPT tracker (categories + hours)
- Forecast finish date recalculation when behind plan
- Better overlap/dependency handling (constraints: "must start no earlier than…")

Phase 3

- Salesforce integration (pull frac info automatically)

- Optimization/suggestions (auto propose reallocations to resolve conflicts)
- Cost modeling (cheapest assignment)

---

10) Open decisions (not blockers, but we should pick defaults)

You left one question unanswered that affects behavior:

How to handle actual frac end being longer/shorter than planned?

I recommend a default rule-set like:

- When **Actual** runs longer than Forecast:
    1. Actual end date extends automatically
    2. Forecast gets a warning: "Actual exceeds forecast by X days"
    3. If chain mode is enabled for that lane, offer a one-click action: **"Push downstream fracs"** vs **"Allow overlap (flag conflicts)"**
- When **Actual** ends earlier:
    1. Actual closes
    2. Offer: "Pull downstream fracs left?" (optional, likely not automatic)

We can make those simple toggles in settings.